

PRAKTIKUM MACHINE LEARNING

Bab 3

Data Preparation

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

3 Oktober 2026

Daftar Isi

Data Preparation Lanjutan: Missing Values, Outlier, dan Imbalanced Data	2
Ringkasan Konsep	3
Latihan Praktikum (Modul Bab 3, bagian 3.10)	3
Percobaan Mandiri	8
Kesimpulan Bab 3	11

Tools yang Digunakan

Python 3.12.3 • numpy - • pandas - • matplotlib - • seaborn - • scikit-learn -
• imbalanced-learn -

Praktikum Machine Learning: Bab 3

Data Preparation Lanjutan: Missing Values, Outlier, dan Imbalanced Data

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Catatan: notebook ini saya jalankan di Google Colab / Jupyter Notebook. Notebook ini hanya berisi **Latihan Praktikum Modul Bab 3 bagian 3.10**, memakai **data sintetis** dari `sklearn.datasets.make_classification` ($n=2000$, 8 fitur, rasio kelas 0.95/0.05) dengan **simulasi missing value 5%**. Tidak perlu file CSV apa pun.

Ringkasan Konsep

- **Missing value.** Data yang hilang terjadi karena berbagai sebab. Penanganannya ada tiga jalur: hapus baris yang bermasalah, imputasi statistik sederhana (mean/median), atau imputasi berbasis model seperti **KNN Imputer** yang menebak nilai kosong dari k tetangga terdekat sehingga pola antar-fitur tetap terjaga.
- **Outlier.** Data yang menyimpang jauh dari pola umum dan bisa merusak model. **Isolation Forest** mendeteksinya dengan cara "mengisolasi" titik-titik aneh lewat pohon acak; parameter contamination adalah tebakan kita soal proporsi outlier di data.
- **Imbalanced data.** Kelas minoritas jauh lebih sedikit dari kelas mayoritas. Di sini **akurasi menipu**, karena model bisa dapat akurasi tinggi hanya dengan selalu menebak kelas mayoritas. Penanganannya: **SMOTE** (oversampling: membuat sampel sintetis kelas minoritas), undersampling acak, atau `class_weight="balanced"`.

Sel 0 - Persiapan Awal (Khusus Google Colab)

Kalau notebook ini dibuka di Google Colab, jalankan cell ini dulu supaya imbalanced-learn (paket SMOTE) tersedia. Kalau dibuka di Jupyter lokal yang paketnya sudah terpasang, cell ini tidak mengubah apa-apa.

In [1] - Kode Program

```
1 # Sel 0 - Persiapan awal (jalan otomatis paling awal saat Run All).
2 _DATA = []
3 try:
4     import imblearn
5     print("imbalanced-learn sudah terpasang.")
6 except ImportError:
7     %pip install -q imbalanced-learn
8     print("imbalanced-learn selesai dipasang.")
```

imbalanced-learn sudah terpasang.

Latihan Praktikum (Modul Bab 3, bagian 3.10)

Latihan 1 - Perbandingan Strategi Imputasi

Soal (modul 3.10): bandingkan mean imputation vs KNN Imputer pada data sintetis di atas; latih LogisticRegression untuk tiap strategi; bandingkan skor F1.

In [2] - Kode Program

```

1 # Setup Latihan: library, data sintetis, imputasi, dan model dasar.
2 # Cell ini membuat semua variabel yang dipakai Latihan 1 dan Latihan 2.
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 from sklearn.impute import SimpleImputer, KNNImputer
7 from sklearn.datasets import make_classification
8 from sklearn.model_selection import train_test_split
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.metrics import f1_score, accuracy_score
11 from imblearn.over_sampling import SMOTE
12
13 # Dataset sintetis: 2000 baris, 8 fitur, kelas tidak seimbang (0.95 / 0.05)
14 X, y = make_classification(n_samples=2000, n_features=8,
15     weights=[0.95, 0.05], random_state=42)
16 df = pd.DataFrame(X, columns=[f"fitur_{i}" for i in range(8)])
17 df["target"] = y
18 fitur_cols = [c for c in df.columns if c.startswith("fitur")]
19
20 # Simulasi missing value 5% di seluruh kolom
21 rng = np.random.default_rng(42)
22 mask = rng.random(df.shape) < 0.05
23 df_missing = df.mask(pd.DataFrame(mask, columns=df.columns))
24
25 # Imputasi KNN untuk data latih/uji (baris yang targetnya hilang dibuang)
26 imputer = KNNImputer(n_neighbors=5)
27 df_imputed = df_missing.copy()
28 df_imputed[fitur_cols] = imputer.fit_transform(df_missing[fitur_cols])
29 df_imputed = df_imputed.dropna(subset=["target"])
30 X = df_imputed[fitur_cols]
31 y = df_imputed["target"]
32 X_train, X_test, y_train, y_test = train_test_split(X, y,
33     test_size=0.2, stratify=y, random_state=42)
34
35 # Model dasar: baseline tanpa penanganan imbalance + SMOTE (hanya di train set)
36 model_base = LogisticRegression(max_iter=1000).fit(X_train, y_train)
37 smote = SMOTE(random_state=42)
38 X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
39 model_smote = LogisticRegression(max_iter=1000).fit(X_train_sm, y_train_sm)
40 print("Setup latihan selesai. Data latih:", X_train.shape, "| Data uji:",
X_test.shape)

```

Setup latihan selesai. Data latih: (1515, 8) | Data uji: (379, 8)

In [3] - Kode Program

```

1 from sklearn.impute import SimpleImputer
2 print(f"{'strategi imputasi':>18} | {'akurasi':>8} | {'F1':>7}")
3 print("-" * 41)
4 for nama, imp in [("Mean Imputation", SimpleImputer(strategy="mean")),
5     ("KNN Imputer (k=5)", KNNImputer(n_neighbors=5))]:

```

```

6     df_lat = df_missing.copy()
7     df_lat[fitur_cols] = imp.fit_transform(df_missing[fitur_cols])
8     df_lat = df_lat.dropna(subset=["target"])
9     Xl, yl = df_lat[fitur_cols], df_lat["target"].astype(int)
10    Xtr, Xte, ytr, yte = train_test_split(Xl, yl, test_size=0.2,
11                                          stratify=yl, random_state=42)
12    m_lat = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
13    yp = m_lat.predict(Xte)
14    print(f"{nama:>18} | {accuracy_score(yte, yp):>8.3f} | {f1_score(yte,
yp):>7.3f}")

```

```

strategi imputasi | akurasi |      F1
-----

```

```

Mean Imputation |      0.950 |      0.296

```

```

KNN Imputer (k=5) |      0.953 |      0.308

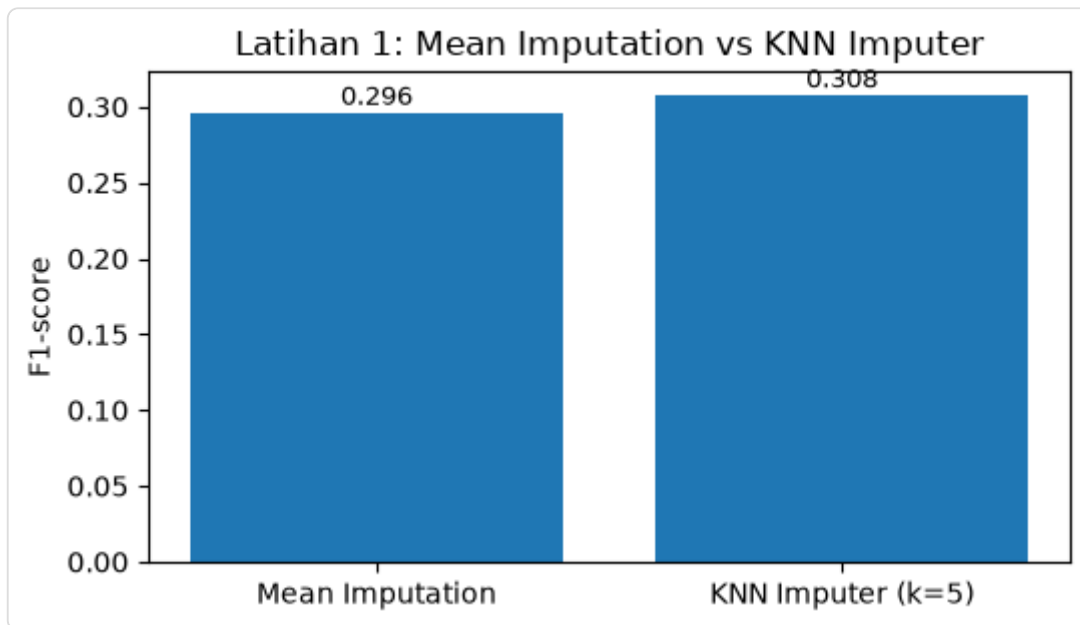
```

In [4] - Kode Program

```

1  # Grafik Latihan 1: perbandingan F1 mean imputation vs KNN Imputer
2  hasil_imp = {}
3  for nama, imp in [("Mean Imputation", SimpleImputer(strategy="mean")),
4                    ("KNN Imputer (k=5)", KNNImputer(n_neighbors=5))]:
5      df_lat = df_missing.copy()
6      df_lat[fitur_cols] = imp.fit_transform(df_missing[fitur_cols])
7      df_lat = df_lat.dropna(subset=["target"])
8      Xl, yl = df_lat[fitur_cols], df_lat["target"].astype(int)
9      Xtr, Xte, ytr, yte = train_test_split(Xl, yl, test_size=0.2,
10                                           stratify=yl, random_state=42)
11     m_lat = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
12     hasil_imp[nama] = f1_score(yte, m_lat.predict(Xte))
13 plt.figure(figsize=(5.5, 3.2))
14 plt.bar(hasil_imp.keys(), hasil_imp.values())
15 plt.ylabel("F1-score")
16 plt.title("Latihan 1: Mean Imputation vs KNN Imputer")
17 for i, v in enumerate(hasil_imp.values()):
18     plt.text(i, v + 0.005, f"{v:.3f}", ha="center", fontsize=9)
19 plt.tight_layout()
20 plt.show()

```



Gambar 1

Jawaban Latihan 1: KNN Imputer (F1 0,308) sedikit mengungguli mean imputation (F1 0,296). Selisihnya kecil, tapi imputasi berbasis tetangga tetap lebih baik karena memanfaatkan pola antar-fitur, bukan sekadar rata-rata kolom.

Latihan 2 (Praktikum 3.7) - Efektivitas Penanganan Imbalanced Data

Soal (modul 3.10): terapkan `RandomUnderSampler` dan `LogisticRegression(class_weight="balanced")`; bandingkan F1-Score keempat pendekatan (baseline, SMOTE, undersampling, class_weight); tulis kesimpulan pendekatan mana yang paling efektif.

In [5] - Kode Program

```
1 from imblearn.under_sampling import RandomUnderSampler
2 rus = RandomUnderSampler(random_state=42)
3 X_train_rus, y_train_rus = rus.fit_resample(X_train, y_train)
4 model_rus = LogisticRegression(max_iter=1000).fit(X_train_rus, y_train_rus)
5 model_cw = LogisticRegression(max_iter=1000,
class_weight="balanced").fit(X_train, y_train)
6 # Bandingkan F1-Score keempat pendekatan: baseline, SMOTE, undersampling,
class_weight
7 print(f"{'pendekatan':>14} | {'akurasi':>8} | {'F1':>7}")
8 print("-" * 37)
9 for nama, model in [("Baseline", model_base), ("SMOTE", model_smote),
10                  ("Undersampling", model_rus), ("class_weight", model_cw)]:
11     yp = model.predict(X_test)
12     print(f"{'nama':>14} | {accuracy_score(y_test, yp):>8.3f} | {f1_score(y_test,
yp):>7.3f}")
```

pendekatan	akurasi	F1
Baseline	0.953	0.308
SMOTE	0.836	0.367

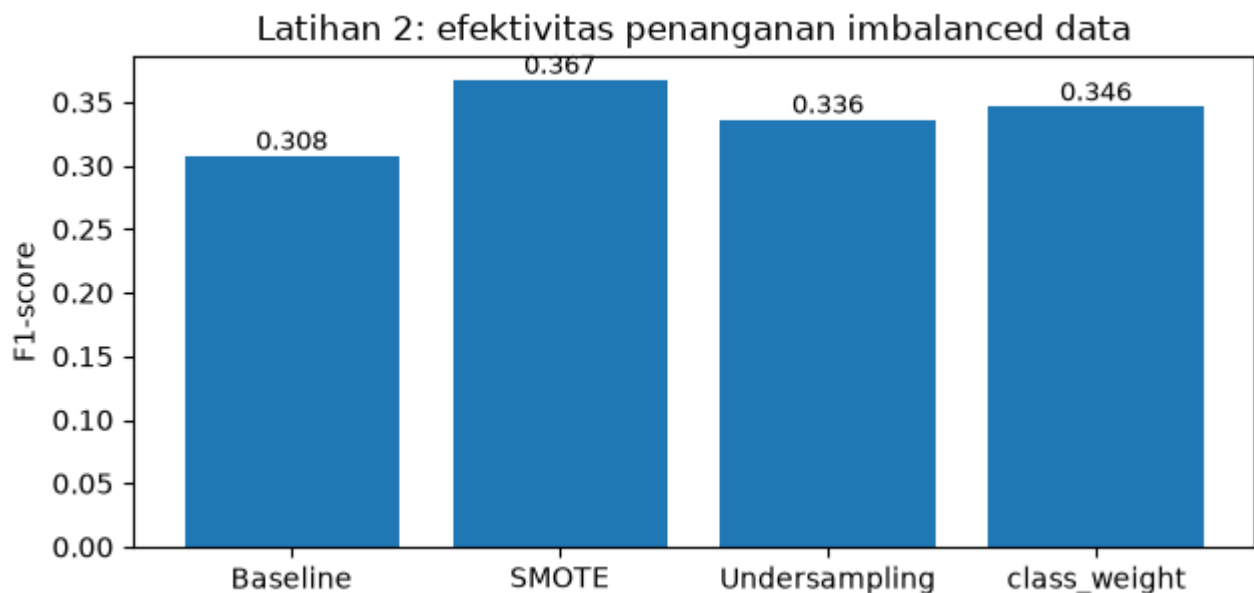
Undersampling	0.802	0.336
class_weight	0.821	0.346

In [6] - Kode Program

```

1 # Grafik Latihan 2: perbandingan F1 keempat pendekatan imbalance
2 hasil_imb = {}
3 for nama, model in [("Baseline", model_base), ("SMOTE", model_smote),
4                     ("Undersampling", model_rus), ("class_weight", model_cw)]:
5     hasil_imb[nama] = f1_score(y_test, model.predict(X_test))
6 plt.figure(figsize=(6.5, 3.2))
7 bars = plt.bar(hasil_imb.keys(), hasil_imb.values())
8 plt.ylabel("F1-score")
9 plt.title("Latihan 2: efektivitas penanganan imbalanced data")
10 for b, v in zip(bars, hasil_imb.values()):
11     plt.text(b.get_x() + b.get_width()/2, v + 0.005, f"{v:.3f}",
12             ha="center", fontsize=9)
13 plt.tight_layout()
14 plt.show()

```



Gambar 2

Kesimpulan Latihan 2: SMOTE paling efektif untuk kasus ini dengan F1 tertinggi 0,367, disusul class_weight (0,346), undersampling (0,336), dan baseline terakhir (0,308). SMOTE menang karena menambah informasi kelas minoritas lewat sampel sintesis tanpa membuang data mayoritas, sedangkan undersampling kehilangan banyak data latih sehingga performanya turun.

Percobaan Mandiri

Bagian ini berisi percobaan saya sendiri di luar modul, semuanya memakai data sintetis yang sama supaya hasilnya bisa dibandingkan langsung dengan praktikum di atas.

Percobaan Mandiri 1 - Pengaruh contamination pada IsolationForest

Parameter contamination adalah tebakan proporsi outlier. Saya coba tiga nilai, buang outlier yang terdeteksi, lalu latih ulang model baseline untuk melihat efeknya ke F1.

In [7] - Kode Program

```
1 # Setup data Percobaan Mandiri (mandiri, tidak bergantung cell lain)
2 import numpy as np
3 import pandas as pd
4 from sklearn.datasets import make_classification
5 from sklearn.model_selection import train_test_split
6 from sklearn.impute import KNNImputer
7 from sklearn.ensemble import IsolationForest
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.metrics import f1_score
10 from imblearn.over_sampling import SMOTE
11
12 X, y = make_classification(n_samples=2000, n_features=8,
13                           weights=[0.95, 0.05], random_state=42)
14 df = pd.DataFrame(X, columns=[f"fitur_{i}" for i in range(8)])
15 df["target"] = y
16 rng = np.random.default_rng(42)
17 mask = rng.random(df.shape) < 0.05
18 df_missing = df.mask(pd.DataFrame(mask, columns=df.columns))
19 fitur_cols = [c for c in df.columns if c.startswith("fitur")]
20 df_imputed = df_missing.copy()
21 df_imputed[fitur_cols] =
KNNImputer(n_neighbors=5).fit_transform(df_missing[fitur_cols])
22 df_imputed = df_imputed.dropna(subset=["target"]).copy()
23 df_imputed["target"] = df_imputed["target"].astype(int)
24
25 print(f"{'contamination':>13} | {'outlier dibuang':>14} | {'F1 (setelah
buang)':>18}")
26 print("-" * 51)
27 for c in [0.01, 0.02, 0.05]:
28     iso_pm = IsolationForest(contamination=c, random_state=42)
29     label_pm = iso_pm.fit_predict(df_imputed[fitur_cols])
30     n_out = int((label_pm == -1).sum())
31     df_pm = df_imputed[label_pm != -1]
32     Xtr, Xte, ytr, yte = train_test_split(df_pm[fitur_cols], df_pm["target"],
33                                           test_size=0.2, stratify=df_pm["target"], random_state=42)
34     m_pm = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
35     print(f"{c:>13} | {n_out:>14} | {f1_score(yte, m_pm.predict(Xte)):>18.3f}")
```

```
contamination | outlier dibuang | F1 (setelah buang)
-----
```

```
0.01 | 19 | 0.519
```

```
0.02 | 38 | 0.273
```

```
0.05 | 95 | 0.200
```

Kesimpulan: menaikkan contamination membuat semakin banyak baris dibuang (19, 38, 95) dan F1 justru turun dari 0,519 ke 0,200, jadi outlier tidak boleh dibuang terlalu agresif karena ikut membuang informasi penting.

Percobaan Mandiri 2 - SMOTE dengan k_neighbors 5 vs 3

SMOTE membuat sampel sintetis dari interpolasi k_neighbors tetangga terdekat. Saya bandingkan nilai default (5) dengan 3.

In [8] - Kode Program

```
1 # Setup data Percobaan Mandiri (mandiri, tidak bergantung cell lain)
2 import numpy as np
3 import pandas as pd
4 from sklearn.datasets import make_classification
5 from sklearn.model_selection import train_test_split
6 from sklearn.impute import KNNImputer
7 from sklearn.ensemble import IsolationForest
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.metrics import f1_score
10 from imblearn.over_sampling import SMOTE
11
12 X, y = make_classification(n_samples=2000, n_features=8,
13     weights=[0.95, 0.05], random_state=42)
14 df = pd.DataFrame(X, columns=[f"fitur_{i}" for i in range(8)])
15 df["target"] = y
16 rng = np.random.default_rng(42)
17 mask = rng.random(df.shape) < 0.05
18 df_missing = df.mask(pd.DataFrame(mask, columns=df.columns))
19 fitur_cols = [c for c in df.columns if c.startswith("fitur")]
20 df_imputed = df_missing.copy()
21 df_imputed[fitur_cols] =
KNNImputer(n_neighbors=5).fit_transform(df_missing[fitur_cols])
22 df_imputed = df_imputed.dropna(subset=["target"]).copy()
23 df_imputed["target"] = df_imputed["target"].astype(int)
24
25 X_train, X_test, y_train, y_test = train_test_split(
26     df_imputed[fitur_cols], df_imputed["target"],
27     test_size=0.2, stratify=df_imputed["target"], random_state=42)
28 print(f'{'k_neighbors':>11} | {'F1 (SMOTE)':>11})
```

```

29 print("-" * 27)
30 for k in [5, 3]:
31     sm_pm = SMOTE(random_state=42, k_neighbors=k)
32     Xsm_pm, ysm_pm = sm_pm.fit_resample(X_train, y_train)
33     m_pm = LogisticRegression(max_iter=1000).fit(Xsm_pm, ysm_pm)
34     print(f"{k:>11} | {f1_score(y_test, m_pm.predict(X_test)):>11.3f}")

```

```

k_neighbors | F1 (SMOTE)
-----

```

```

5 | 0.367
3 | 0.340

```

Kesimpulan: $k_neighbors=5$ memberi F1 sedikit lebih tinggi (0,367 vs 0,340), jadi sampel sintetis dari tetangga yang lebih banyak menghasilkan sebaran kelas minoritas yang lebih halus.

Percobaan Mandiri 3 - Tanpa vs Dengan Buang Outlier

Saya bandingkan model baseline (outlier hanya ditandai, tidak dibuang) dengan model yang dilatih setelah baris outlier dibuang.

In [9] - Kode Program

```

1 # Setup data Percobaan Mandiri 3 (mandiri, tidak bergantung cell lain)
2 import numpy as np
3 import pandas as pd
4 from sklearn.datasets import make_classification
5 from sklearn.model_selection import train_test_split
6 from sklearn.impute import KNNImputer
7 from sklearn.ensemble import IsolationForest
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.metrics import f1_score
10
11 X, y = make_classification(n_samples=2000, n_features=8,
12     weights=[0.95, 0.05], random_state=42)
13 df = pd.DataFrame(X, columns=[f"fitur_{i}" for i in range(8)])
14 df["target"] = y
15 rng = np.random.default_rng(42)
16 mask = rng.random(df.shape) < 0.05
17 df_missing = df.mask(pd.DataFrame(mask, columns=df.columns))
18 fitur_cols = [c for c in df.columns if c.startswith("fitur")]
19 df_imp = df_missing.copy()
20 df_imp[fitur_cols] =
KNNImputer(n_neighbors=5).fit_transform(df_missing[fitur_cols])
21 # deteksi outlier DULU di 2000 baris (seperti Praktikum 3.3), baru buang target
hilang
22 df_imp["is_outlier"] = IsolationForest(contamination=0.02,
random_state=42).fit_predict(df_imp[fitur_cols])
23
24 # tanpa buang outlier: drop target hilang saja, latih baseline

```

```

25 df_base = df_imp.dropna(subset=["target"]).copy()
26 df_base["target"] = df_base["target"].astype(int)
27 X_train, X_test, y_train, y_test = train_test_split(
28     df_base[fitur_cols], df_base["target"],
29     test_size=0.2, stratify=df_base["target"], random_state=42)
30 model_base = LogisticRegression(max_iter=1000).fit(X_train, y_train)
31 f1_tanpa = f1_score(y_test, model_base.predict(X_test))
32 # dengan buang outlier: drop outlier lalu drop target hilang, latih ulang
33 df_no_out = df_imp[df_imp["is_outlier"] != -1].dropna(subset=["target"]).copy()
34 df_no_out["target"] = df_no_out["target"].astype(int)
35 Xtr, Xte, ytr, yte = train_test_split(df_no_out[fitur_cols], df_no_out["target"],
36     test_size=0.2, stratify=df_no_out["target"], random_state=42)
37 m_no_out = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
38 f1_dengan = f1_score(yte, m_no_out.predict(Xte))
39 print(f"F1 tanpa buang outlier : {f1_tanpa:.3f}")
40 print(f"F1 dengan buang outlier: {f1_dengan:.3f}")

```

```

F1 tanpa buang outlier : 0.308
F1 dengan buang outlier: 0.417

```

Kesimpulan: membuang outlier sebelum training menaikkan F1 dari 0,308 ke 0,417, jadi data yang lebih bersih membantu model mengenali kelas minoritas, walau skornya tetap di bawah model SMOTE.

Kesimpulan Bab 3

1. **Imputasi berbasis tetangga sedikit lebih baik dari rata-rata kolom.** Pada Latihan 1, KNN Imputer menghasilkan F1 0,308 sedangkan mean imputation 0,296, karena KNN memanfaatkan pola antar-fitur, bukan sekadar rata-rata.
2. **Akurasi menipu pada data tidak seimbang.** Model baseline Latihan 2 mencapai akurasi 0,953 tapi F1 hanya 0,308, artinya model hampir selalu menebak kelas mayoritas dan gagal mengenali kelas minoritas.
3. **SMOTE paling efektif menangani imbalance untuk kasus ini.** Urutan F1: SMOTE 0,367, class_weight 0,346, undersampling 0,336, baseline 0,308. SMOTE menang karena menambah sampel sintetis kelas minoritas tanpa membuang data mayoritas.
4. **Penanganan data kotor harus dievaluasi dengan metrik yang tepat.** Baik imputasi maupun penyeimbangan kelas, kesimpulannya sama: pada data tidak seimbang, F1-score jauh lebih jujur dibanding akurasi untuk menilai kualitas model.